

Esercizio: Sistema di Gestione dei Documenti

Scenario:

Progettare un sistema di gestione dei documenti per un'azienda. Il sistema deve supportare diversi tipi di documenti (PDF, Word, Excel) e deve essere in grado di salvare, aprire e visualizzare i documenti. Inoltre, il sistema deve permettere l'integrazione con diverse piattaforme di storage (locale, cloud). Utilizzeremo il pattern Factory Method per la creazione dei diversi tipi di documenti e il pattern Bridge per separare l'astrazione (operazioni sui documenti) dall'implementazione (piattaforme di storage).

Obiettivo dell'Esercizio

Comprendere l'utilizzo combinato dei pattern Factory Method e Bridge per creare un sistema flessibile e manutenibile che gestisca diversi tipi di documenti e interagisca con diverse piattaforme di storage, assicurando che il sistema sia estensibile, permettendo l'aggiunta di nuovi tipi di documenti e nuove piattaforme di storage senza modificare il codice esistente. Più precisamente:

- Creare istanze delle factory per i diversi tipi di documenti.
- Utilizzare le factory per creare documenti.
- Creare istanze delle piattaforme di storage.
- Utilizzare il pattern Bridge per gestire le operazioni sui documenti con le diverse piattaforme di storage.
- Eseguire operazioni sui documenti come apertura, salvataggio e visualizzazione, utilizzando il sistema progettato.

Istruzioni:

1. Interfacce per i Documenti e le Piattaforme di Storage

- Definire un'interfaccia o classe astratta `Document` con i metodi `open()`, `save()` e `view()`.
- Definire un'interfaccia o classe astratta `Storage` con i metodi `store(Document doc)` e `retrieve(String docName)`.

2. Implementazioni Concrete dei Documenti

- Creare classi concrete per `PdfDocument`, `WordDocument` e `ExcelDocument` che implementano l'interfaccia o estendono la classe astratta `Document`.
- Assicurarsi che ogni classe concreta implementi i metodi per aprire, salvare e visualizzare i documenti.

3. Implementazioni Concrete delle Piattaforme di Storage

- Creare classi concrete per `LocalStorage` e `CloudStorage` che implementano l'interfaccia o estendono la classe astratta `Storage`.
- Assicurarsi che ogni classe concreta implementi i metodi per memorizzare e recuperare i documenti.

4. Factory Method per la Creazione dei Documenti

- Definire un'interfaccia `DocumentFactory` con il metodo `createDocument()`.
- Creare classi concrete `PdfDocumentFactory`, `WordDocumentFactory` e `ExcelDocumentFactory` che implementano l'interfaccia `DocumentFactory`.
- Assicurarsi che ogni factory crei e ritorni un'istanza del rispettivo tipo di documento.

5. Bridge per Separare le Operazioni sui Documenti dalle Piattaforme di Storage

- Definire una classe astratta `DocumentHandler` che contenga un riferimento a un'istanza di `Storage` e metodi astratti `saveDocument(Document doc)` e `openDocument(String docName)`.
- Creare una classe concreta `DocumentHandlerImpl` che estenda `DocumentHandler` e implementi i metodi astratti, utilizzando le operazioni di storage per memorizzare e recuperare i documenti.

6. Scenario di utilizzo del Sistema (*Main*):

1. Creare una factory per documenti PDF e un'istanza di storage locale.
2. Utilizzare la factory per creare un documento PDF.
3. Utilizzare il Bridge per salvare il documento PDF nel storage locale.
4. Creare una factory per documenti Word e un'istanza di storage cloud.
5. Utilizzare la factory per creare un documento Word.
6. Utilizzare il Bridge per salvare il documento Word nel storage cloud.
7. Utilizzare il Bridge per aprire un documento Excel dal storage locale.

Consegna

- Creare il file **EsercizioPattern-CognomeNome.zip** con i file .java relativi al progetto
- copiare il file nel path *EsercizioStudenti* relativo al giorno di lezione